Automated User Management System

System and Network administration Project Report

Project repository:

https://github.com/ArthurAipov/Automated-User-Management-System

Project demo video:

https://disk.yandex.ru/i/-gsdi1H06OXPCQ

Team members and responsibilities:

Aipov Arthur : main developer of the project, including Bash automation, user management logic, systemd configuration, testing materials, and project documentation

Ruslan Ustinov : project idea initiator, contribution to solution design and testing workflow, demo video preparation, report writing, and final presentation materials

---

I. Goal / Tasks Of The Project

The goal of this project was to build a Bash-based automated Linux user management system for Ubuntu/Debian environments. The system was designed to reduce repetitive manual administration work when many temporary accounts must be created, updated, expired, and later removed.

Manual user management is error-prone: an administrator can forget to assign the correct group, set unsafe home directory permissions, miss an expiry date, or accidentally delete the wrong account. This project solves that problem by using a repeatable CSV-driven workflow, strict validation, logging, dry-run mode, and a dedicated management group.

Target Use Case

The solution targets a university or laboratory environment where administrators need to manage student, internship, lab, or research accounts on a Linux server. Each account has a username, primary group, home directory, and expiry date. The proof of concept was tested on an Ubuntu 24.04.4 LTS cloud server with root access, SSH, standard Linux account tools, and systemd.

Major Completed Tasks

- Created a CSV-driven workflow for creating and updating Linux users.

- Implemented root privilege checks to avoid partial execution.
- Added a dedicated safety group named `managed_users`.
- Implemented `--dry-run` mode for safe previews.
- Added logging to `/var/log/user_manager.log`.
- Automated SSH key generation for managed users.
- Applied password and account expiry policies using `chage`.
- Implemented cleanup of expired managed accounts.
- Added a systemd service and timer for weekly cleanup.
- Prepared testing commands, report materials, README documentation, and demo guides.

II. Execution Plan / Methodology

The planned methodology was to keep user account data separate from automation logic. User records are stored in a simple CSV file, while the Bash script reads, validates, and applies this data to the Linux system. This makes the process repeatable and easier to audit.

Planned Infrastructure

The intended infrastructure is a safe Ubuntu/Debian VM or temporary cloud server. The project should not be tested directly on a production machine because it creates and deletes real Linux users. The demo workflow used SSH access from a macOS terminal to an Ubuntu cloud server, which provided a realistic environment with `useradd`, `usermod`, `userdel`, `chage`, `ssh-keygen`, and systemd.

CSV-Based Workflow

The input file is `users.csv`. It contains four columns: `username`, `group`, `home`, and `expiry`. Each row defines one Linux account that should be created or updated.

```
username,group,home,expiry
student01,students,/home/student01,2026-12-31
intern01,interns,/home/intern01,2026-09-30
```

Component Roles

- `users.csv`: source of user account data.
- `user_manager.sh`: main automation logic.
- `/var/log/user_manager.log`: audit trail of actions.
- `user-cleanup.service`: systemd unit that runs expired account cleanup.

- `user-cleanup.timer`: weekly systemd schedule.
- Ubuntu VM or cloud server: isolated proof-of-concept environment.

Workflow Scheme

```
users.csv
   -> user_manager.sh --create
   -> validate CSV rows
   -> create groups and users
   -> configure home directories, SSH keys, and chage policy
   -> write actions to /var/log/user_manager.log

systemd timer
   -> user-cleanup.service
   -> user_manager.sh --cleanup-expired
   -> find expired users inside managed_users
   -> delete only eligible managed accounts
   -> write cleanup results to the log
```

Safety Mechanisms

Several safety mechanisms were included from the planning stage. The `--dry-run` option previews actions without changing the system. The `managed_users` group prevents cleanup from touching unrelated or system accounts. Logging records all important operations, especially dangerous actions such as user deletion. A root check prevents inconsistent execution when the script is started without required privileges.

Assumptions

The project assumes a non-production Ubuntu/Debian test machine, a simple CSV format without quoted commas, and existing users outside `managed_users` must not be modified. It also assumes that the server used for testing is temporary or safe for administrative experiments.

III. Development Of Solution / Tests As Proof Of Concept

Main Files

- `scripts/user_manager.sh`: main account management script.
- `scripts/prepare_demo_server.sh`: prepares the Ubuntu/Debian server for demo.
- `users.csv`: sample input with 11 users.
- `systemd/user-cleanup.service`: oneshot cleanup service.
- `systemd/user-cleanup.timer`: weekly systemd timer.

- `tests/test_commands.md`: manual testing commands.
- `reports/test_report.md`: test report template.
- `README.md`: usage, installation, testing, and cleanup instructions.

Implementation Details

The main script parses the CSV file line by line and skips the header row. For every data row, it validates the username, group name, home directory path, and expiry date. Invalid rows are logged and skipped, while valid rows are processed.

The script creates the `managed_users` group and the required primary groups. New users are created with `/bin/bash`, the requested home directory, primary group, management group, and expiry date. Existing users are updated only if they already belong to `managed_users`; this prevents the tool from changing unrelated accounts.

Home directories are configured with owner, group, and `700` permissions. SSH keys are generated as Ed25519 keys only when missing. The `.ssh` directory receives `700`, the private key receives `600`, and the public key receives `644`. The password policy is applied with `chage -M 90 -W 7 -E expiry`.

Core Commands Used By The Script

```
useradd -m -d /home/student01 -s /bin/bash -g students -G
   managed_users -e 2026-12-31 student01
chage -M 90 -W 7 -E 2026-12-31 student01
ssh-keygen -t ed25519 -f /home/student01/.ssh/id_ed25519 -N ""
chmod 700 /home/student01
chmod 600 /home/student01/.ssh/id_ed25519
chmod 644 /home/student01/.ssh/id_ed25519.pub
```

Systemd Automation

The systemd service executes `/usr/local/bin/user_manager.sh --cleanup-expired`. The timer uses `OnCalendar=weekly`, so cleanup runs automatically every week. It is enabled with `systemctl enable --now user-cleanup.timer`. This converts the script from a manual administrative tool into a scheduled maintenance process.

Proof-Of-Concept Testing

The proof of concept was tested on an Ubuntu 24.04.4 LTS cloud server. The following tests were used to prove that the solution works correctly:

- **Dry-run create**: `./scripts/user_manager.sh --create users.csv --dry-run` printed planned actions without changing accounts.
- **Real create**: `./scripts/user_manager.sh --create users.csv` created or updated all users from CSV.
- **User verification**: `id student01` showed that `student01` exists and belongs

to `students` and `managed_users`.

- **Managed group verification**: `getent group managed_users` showed all users controlled by the project.
- **Account record verification**: `getent passwd student01` showed `/home/student01` and `/bin/bash`.
- **Home permissions**: `stat -c '%U %G %a %n' /home/student01` returned owner `student01`, group `students`, and mode `700`.
- **SSH key verification**: `ls -la /home/student01/.ssh` showed `id_ed25519` and `id_ed25519.pub`.
- **SSH permissions**: the expected modes were `700` for `.ssh`, `600` for the private key, and `644` for the public key.
- **Password expiry**: `chage -l student01` showed maximum password age `90`, warning period `7`, and account expiry from the CSV file.
- **Logging**: `tail /var/log/user_manager.log` showed create, update, SSH, permission, and `USER_READY` actions.
- **Idempotency**: running create again did not duplicate users; the log showed update actions and existing SSH keys.
- **Systemd timer**: after installation, `systemctl status user-cleanup.timer` showed the timer as active and waiting.
- **Expired user cleanup**: an expired user `expired01` with date `2020-01-01` was created, previewed with dry-run cleanup, then removed by real cleanup.

Cleanup Test Commands

```
printf 'username,group,home,expiry\nexpired01,students,/home/
    expired01,2020-01-01\n' > /tmp/expired-users.csv
./scripts/user_manager.sh --create /tmp/expired-users.csv
id expired01
./scripts/user_manager.sh --cleanup-expired --dry-run
./scripts/user_manager.sh --cleanup-expired
getent passwd expired01 || echo 'expired01 removed successfully'
```

Observed Evidence

The observed evidence confirmed that the project worked as intended. The server was Ubuntu 24.04.4 LTS. The user `student01` had a UID, the primary group `students`, and membership in `managed_users`. The home directory had `700` permissions. SSH keys existed with `600`/`644` permissions. `chage` showed maximum password age `90`, warning period `7`, and account expiry on December 31, 2026. The systemd timer became active and waiting. The expired user `expired01` was deleted successfully.

Limitations

The proof of concept intentionally stays simple. The CSV parser does not support quoted commas. There is no graphical interface. There is no unit test framework or ShellCheck CI integration yet. SSH private keys are generated locally inside user home directories, which would need stronger review for production. The cleanup schedule is fixed weekly unless the timer file is edited.

IV. Difficulties Faced And New Skills Acquired

Difficulties

The main difficulty was designing the project so that it performs real Linux administration tasks while remaining safe in a test environment. User deletion is a dangerous operation, so cleanup had to be restricted to accounts that belong to `managed_users`. This design prevents accidental deletion of unrelated accounts.

Another challenge was making the script idempotent. Re-running the same command should not create duplicate users or break existing accounts. Existing users are updated only when they are already managed by the project.

The project also required careful handling of root-only operations, file ownership, permissions, SSH key generation, `chage` behavior, systemd unit installation, and clean demo server preparation. During cleanup testing, harmless system warnings such as missing mail spool files also had to be understood and explained.

New Skills Acquired

- Bash scripting with functions, strict mode, validation, and command-line arguments.
- Linux user and group administration using `useradd`, `usermod`, `userdel`, and `groupadd`.
- Secure file permissions with `chmod`, `chown`, and `install`.
- Account expiry management using `chage`.
- SSH key automation with `ssh-keygen`.
- Logging and audit trail design.
- Dry-run safety design.
- systemd service and timer automation.
- Testing in an Ubuntu cloud server environment.
- Technical documentation and demo planning.

V. Conclusion, Contemplations, And Judgment

The project achieved its main objective: it produced a working automated user management system for Ubuntu/Debian. The solution can create and update accounts from a CSV file, configure groups, home directories, SSH keys, password expiry policies, log all actions, and safely clean expired managed accounts. Weekly cleanup is automated through systemd.

The solution is useful because it reduces repetitive administrative work and makes account creation more predictable. The dedicated `managed_users` group is an important safety decision because it limits destructive cleanup actions to accounts explicitly controlled by the project. The log file improves auditability, and dry-run mode makes the tool easier to test before applying real changes.

Future Improvements

- Use a stronger CSV parser with support for quoted fields.
- Add an external configuration file for paths, shell, password policy, and schedule.
- Add structured JSON or CSV reporting output.
- Add optional email notifications after cleanup.
- Integrate log rotation.
- Add ShellCheck and automated tests in CI.
- Improve SSH key handling and distribution policy.
- Make the systemd cleanup schedule configurable.

Final Judgment

Overall, this project is a complete and functional proof of concept. It demonstrates Bash automation, Linux account management, secure file permissions, logging, safety mechanisms, and systemd scheduling in a realistic Ubuntu/Debian environment. With additional testing and configuration improvements, the same design could be expanded into a more production-ready administrative tool.